

```

In [3]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px

# Load the dataset
file_path = "Indian_food.csv"
df = pd.read_csv(file_path)

# Step 1: Replace '-1' in 'state' with the mode
mode_state = df[df['state'] != '-1']['state'].mode()[0]
df['state'] = df['state'].replace('-1', mode_state)

# Step 2: Fill corresponding region where 'state' was replaced
mode_region = df[df['state'] == mode_state]['region'].mode()[0]
df['region'] = df['region'].replace('-1', mode_region)

# Step 3: Replace '-1' in 'cook_time' with the mean
mean_cook_time = df[df['cook_time'] != -1]['cook_time'].astype(float).mean()
df['cook_time'] = df['cook_time'].replace(-1, mean_cook_time)

# 1. Pie Chart: Most preferred flavors
flavor_counts = df['flavor_profile'].value_counts()
plt.figure(figsize=(8, 6))
plt.pie(flavor_counts, labels=flavor_counts.index, autopct='%1.1f%%', col
plt.title("Most Preferred Flavors by Customers")
plt.show()

# 2. Bar Chart: Most requested courses
course_counts = df['course'].value_counts()
plt.figure(figsize=(10, 6))
ax = sns.barplot(x=course_counts.index, y=course_counts.values, palette="

# Annotate values on top of bars
for i, v in enumerate(course_counts.values):
    ax.text(i, v + 1, str(v), ha='center', fontsize=12)

plt.title("Most Requested Courses by Customers", fontsize=14)
plt.xlabel("Course", fontsize=12)
plt.ylabel("Count", fontsize=12)
plt.xticks(rotation=45)
plt.show()

# 3. Grouping and plotting
veg_counts = df.groupby(['region', 'Diet']).size().unstack()

# Plot the stacked bar chart with custom colors
veg_counts.plot(kind='bar', stacked=True, figsize=(10, 6), color=['green'

# Titles and labels
plt.title("Comparison of Vegetarian and Non-Vegetarian Orders by Region")
plt.xlabel("Region")
plt.ylabel("Count")
plt.legend(title="Diet", labels=["Vegetarian", "Non-Vegetarian"])
plt.show()

# 4. Box Plot: Cooking Time Distribution

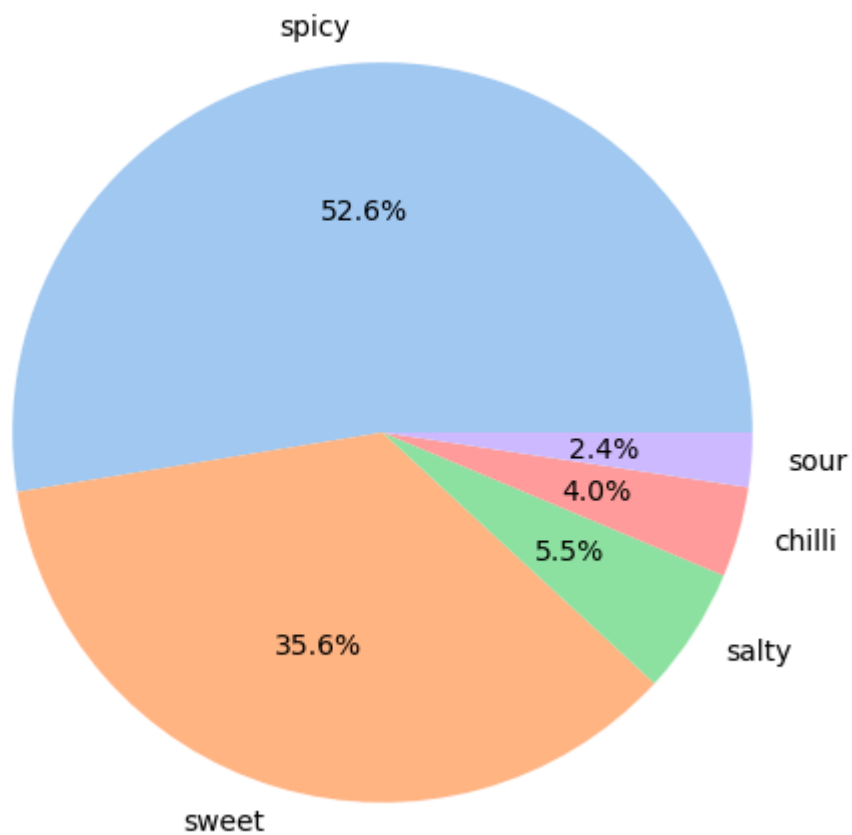
```

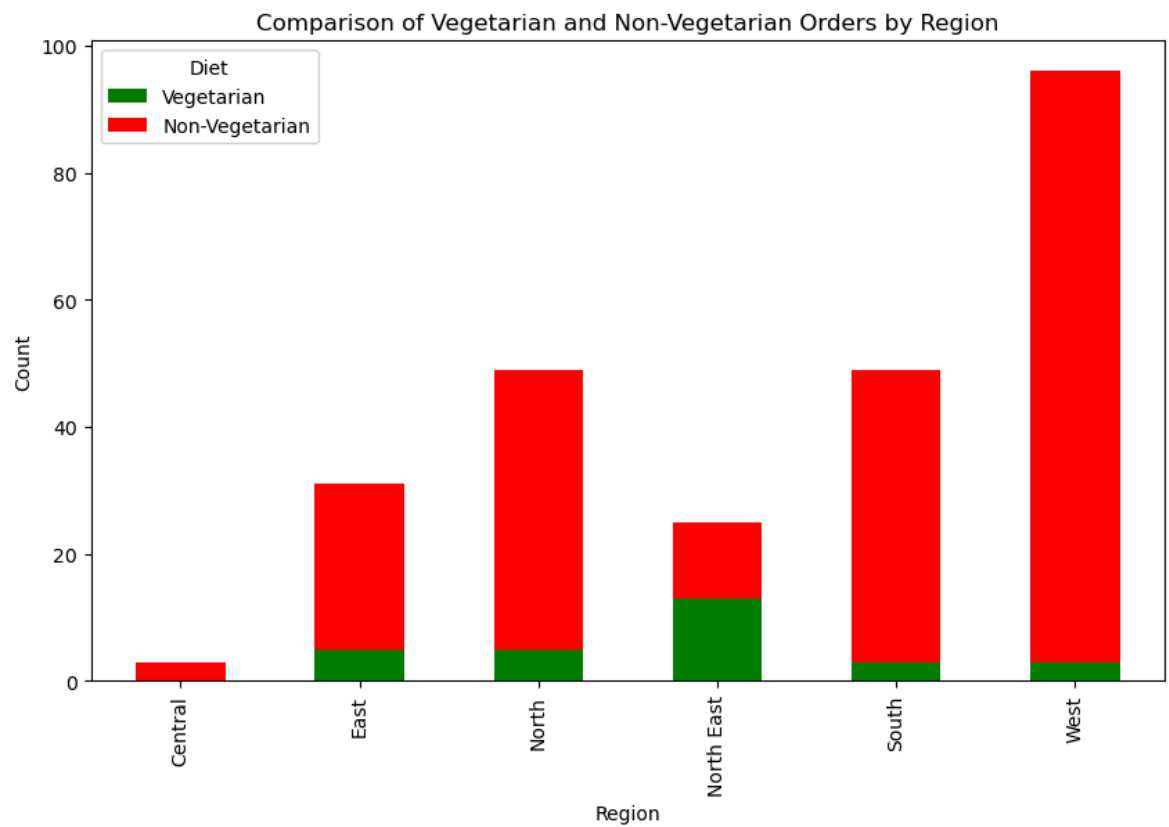
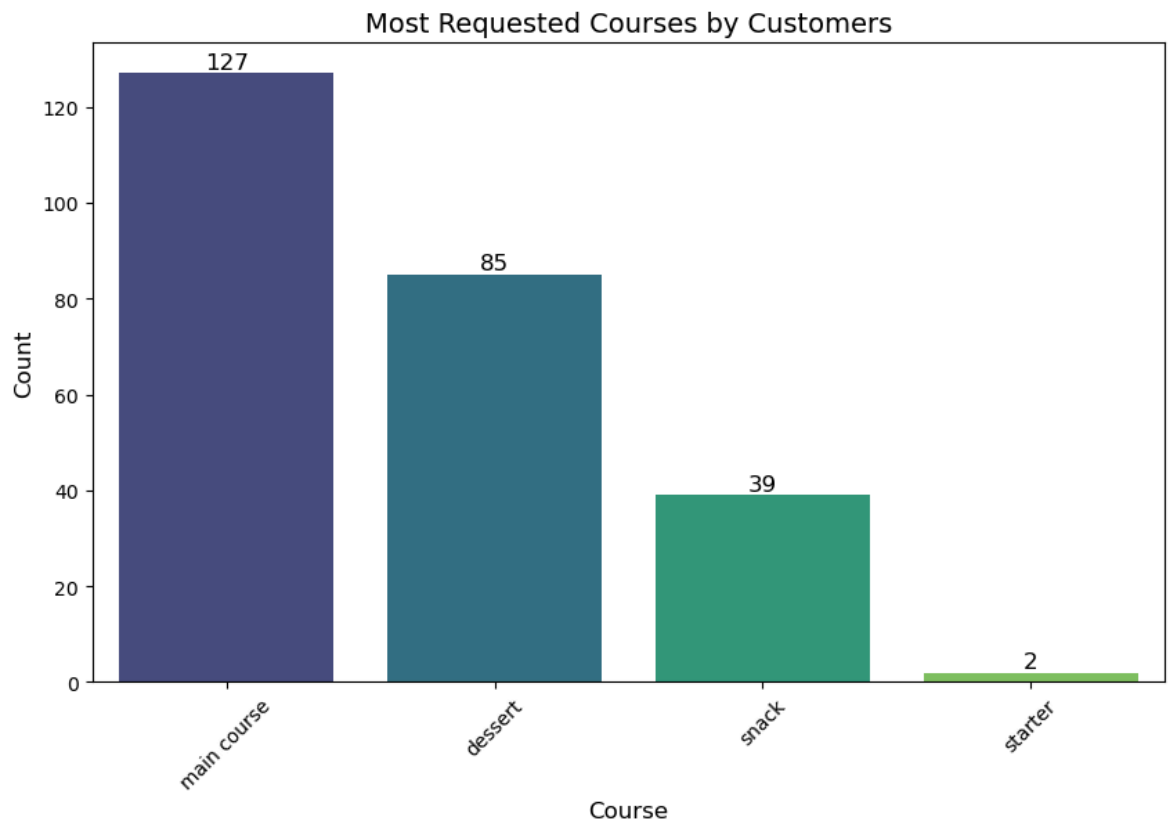
```
plt.figure(figsize=(10, 6))
sns.boxplot(y=df['cook_time'], palette="coolwarm")
plt.title("Distribution of Cooking Time with Outliers")
plt.ylabel("Cooking Time (minutes)")
plt.show()

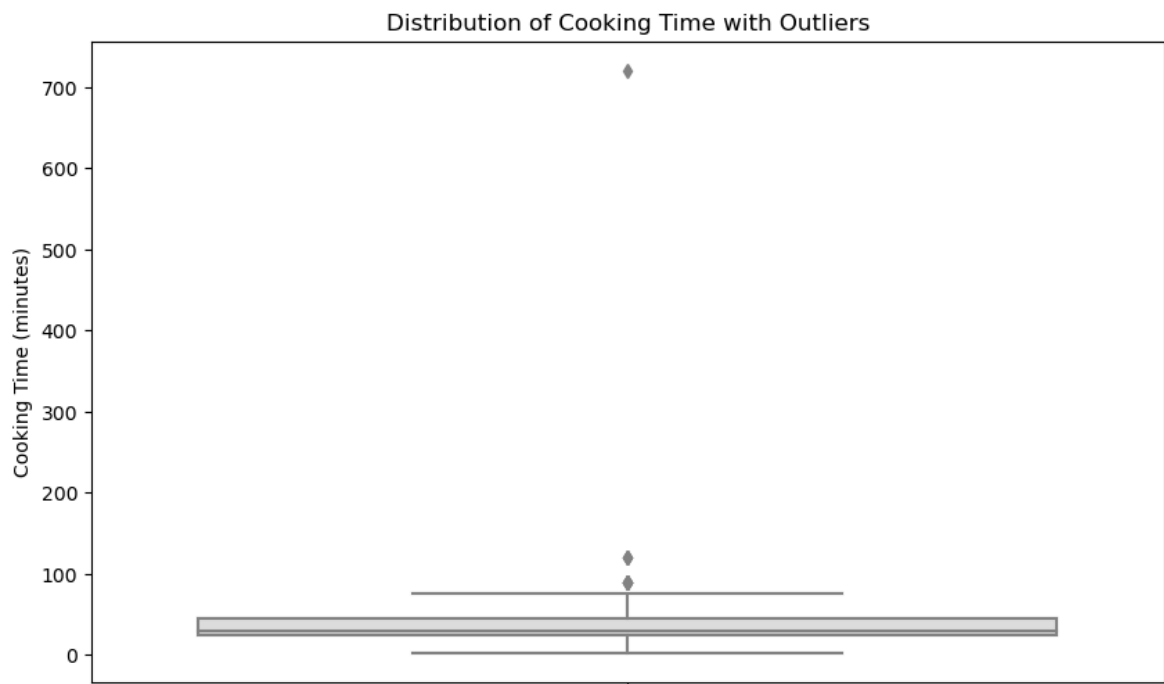
# Checking for outliers
Q1 = df['cook_time'].quantile(0.25)
Q3 = df['cook_time'].quantile(0.75)
IQR = Q3 - Q1

outliers = df[(df['cook_time'] < Q1 - 1.5 * IQR) | (df['cook_time'] > Q3 + 1.5 * IQR)]
print(f'Number of outliers detected: {len(outliers)}')
```

Most Preferred Flavors by Customers







Number of outliers detected: 11

```
In [29]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load the data
df = pd.read_csv('Billionaires_Stats.csv')

# a) Filter billionaires aged 30 or older and create scatter plot
df_age_30 = df[df['age'] >= 30]

plt.figure(figsize=(8, 6))
plt.scatter(df_age_30['age'], df_age_30['finalWorth'], alpha=0.6, color='red')
plt.title('age vs finalWorth of Billionaires')
plt.xlabel('age')
plt.ylabel('finalWorth (in billions)')
plt.show()

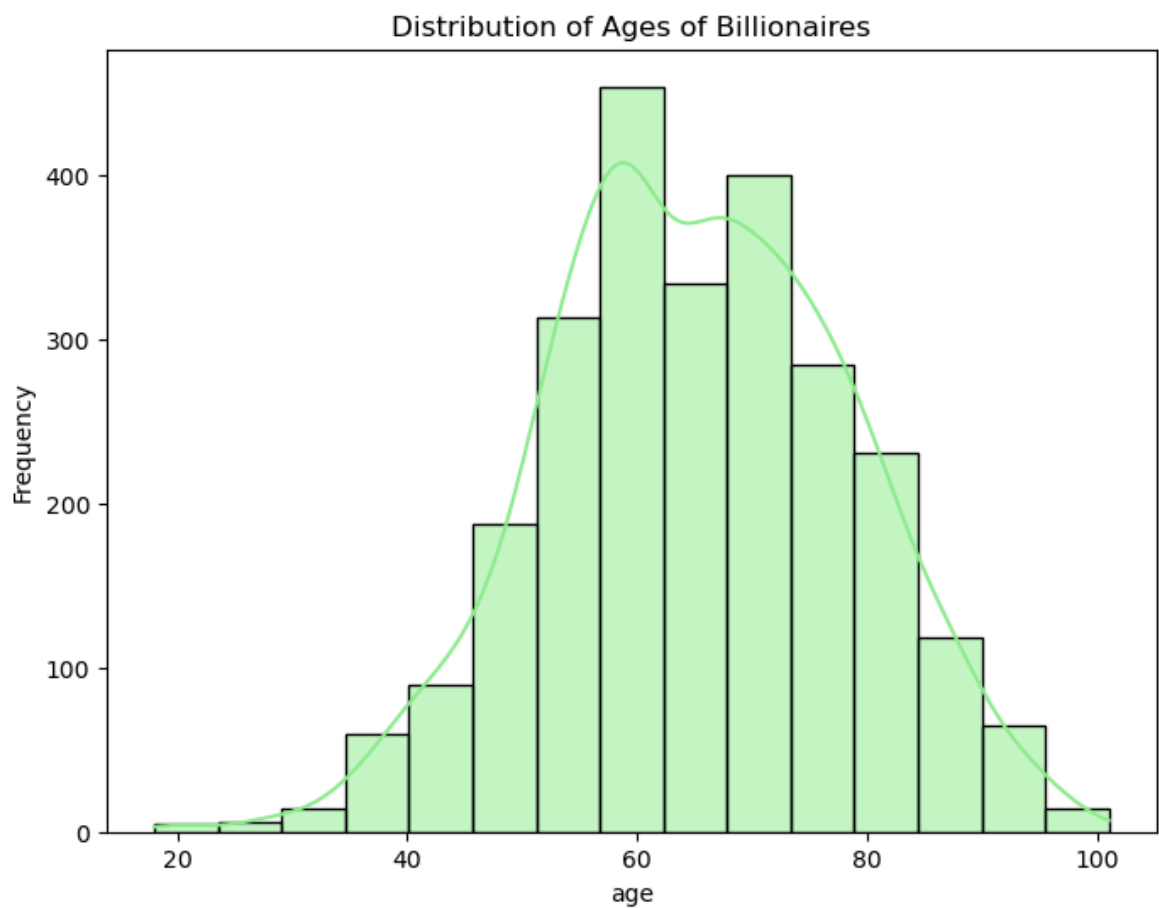
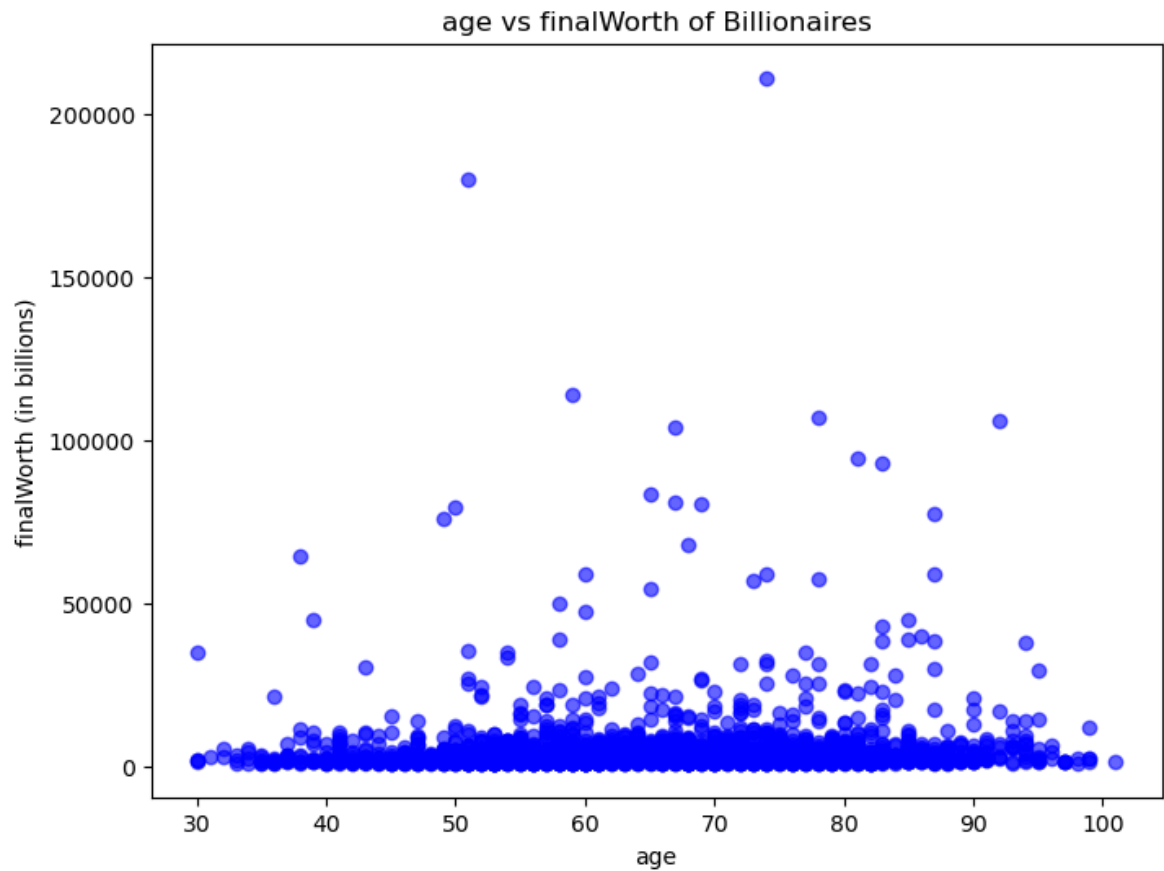
# b) Create a histogram of ages
plt.figure(figsize=(8, 6))
sns.histplot(df['age'], bins=15, kde=True, edgecolor='black', color='lightcoral')
plt.title('Distribution of Ages of Billionaires')
plt.xlabel('age')
plt.ylabel('Frequency')
plt.show()

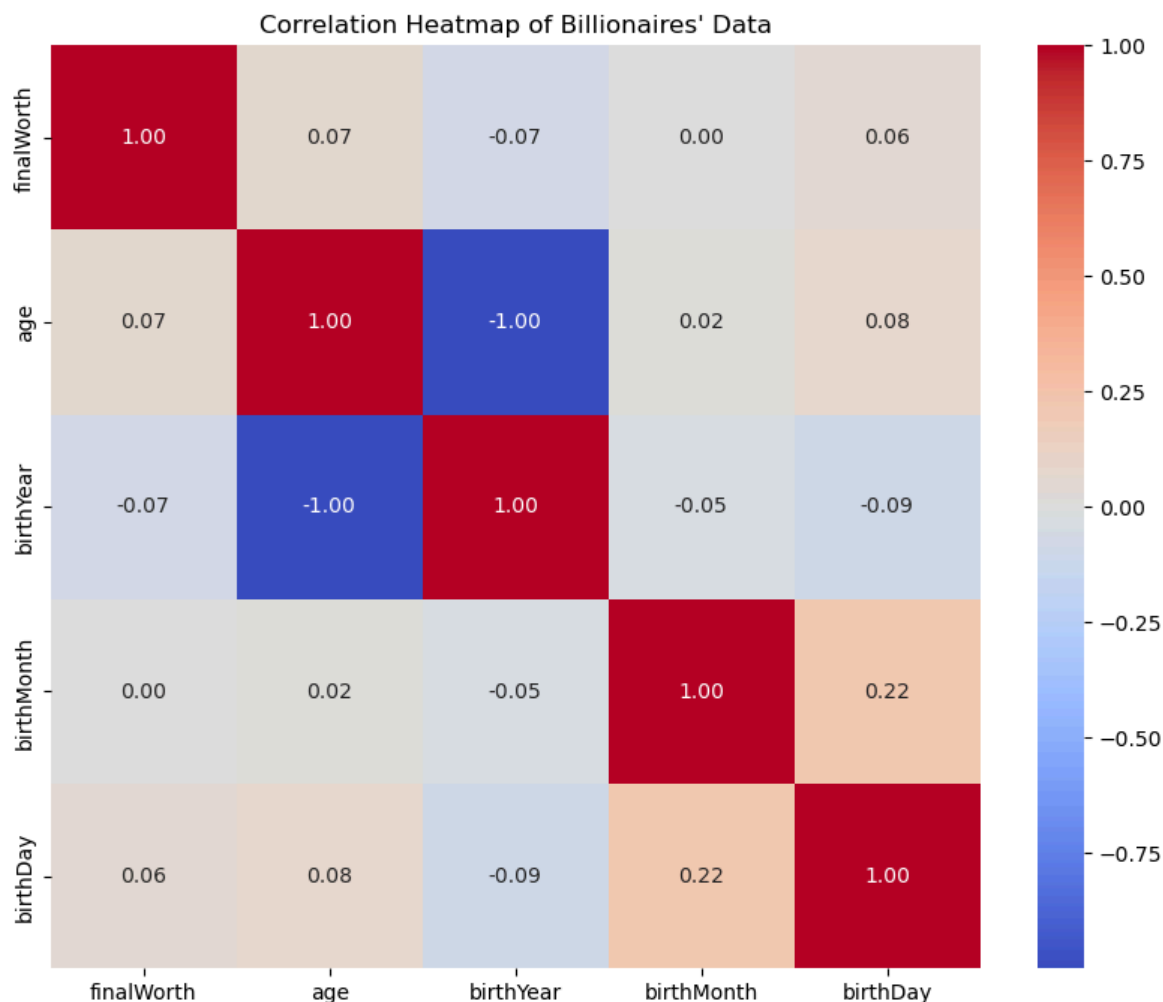
# c) Create a heatmap to visualize the correlation matrix
numeric_df = df.select_dtypes(include=['number'])
correlation_matrix = numeric_df.corr()

plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f', cbar=True)
plt.title('Correlation Heatmap of Billionaires\' Data')
plt.show()

# d) Save the heatmap figure
# Save the heatmap to a file in the current working directory
plt.savefig('heatmap_correlation.png')
```

```
# Save the scatter plot as well, if needed
#plt.savefig('scatter_plot_age_net_worth.png')
# Print confirmation message
print("The heatmap has been saved as 'heatmap_correlation.png' in the cur
```





The heatmap has been saved as 'heatmap_correlation.png' in the current working directory.

<Figure size 640x480 with 0 Axes>

In [6]: *# 5 types of charts are given below:*
#1.Bar Chart (Seaborn): Used to compare categorical data (e.g., sales by
#2.Pie Chart (Plotly): Displays proportions of a whole (e.g., market shar
#3.Scatter Plot (Matplotlib): Shows relationships between two numerical v
#4.Histogram (Seaborn): Displays the distribution of a dataset (e.g., tes
#5: Boxplot (Seaborn): Used for summarizing data distributions, detecting

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load the dataset
file_path = "Zomato-data.csv"
df = pd.read_csv(file_path)

# Step 1: Data Cleaning
df.drop_duplicates(inplace=True)
df.dropna(inplace=True)

print("Data after cleaning:")
print(df.info())

# Create a numeric rate column (ignoring non-numeric entries like "NEW")
df['numeric_rate'] = pd.to_numeric(df['rate'].str.split('/').str[0], erro
```

Step 2: Pie Chart

```
column_for_pie = 'listed_in(type)'\nif column_for_pie in df.columns:\n    pie_data = df[column_for_pie].value_counts()\n    plt.figure(figsize=(8, 6))\n    pie_data.plot(kind='pie', autopct='%1.1f%%', startangle=90)\n    plt.title('Pie Chart of ' + column_for_pie)\n    plt.ylabel('')\n    plt.show()
```

Step 3: Bar Chart

```
column_for_bar = 'name'\nif 'numeric_rate' in df.columns and column_for_bar in df.columns:\n    bar_data = df.groupby(column_for_bar)['numeric_rate'].mean().nlargest\n    plt.figure(figsize=(10, 6))\n    bar_data.plot(kind='bar', color='skyblue')\n    plt.title('Top 10 Restaurants by Average Rating')\n    plt.xlabel('Restaurant Name')\n    plt.ylabel('Average Rating')\n    plt.xticks(rotation=45)\n    plt.tight_layout()\n    plt.show()
```

Step 4: Scatter Plot

```
if 'numeric_rate' in df.columns and 'approx_cost(for two people)' in df.c\n    plt.figure(figsize=(8, 6))\n    sns.scatterplot(data=df, x='approx_cost(for two people)', y='numeric_\n    plt.title('Scatter Plot: Approx Cost (for two people) vs. Rating')\n    plt.xlabel('Approx Cost (for two people)')\n    plt.ylabel('Rating')\n    plt.show()
```

Step 5: Histogram

```
if 'approx_cost(for two people)' in df.columns:\n    plt.figure(figsize=(8, 6))\n    df['approx_cost(for two people)'].plot(kind='hist', bins=20, color='c\n    plt.title('Histogram of Approx Cost (for two people)')\n    plt.xlabel('Approx Cost (for two people)')\n    plt.ylabel('Frequency')\n    plt.tight_layout()\n    plt.show()
```

Step 6: Boxplot

```
# plt.figure(figsize=(12, 6))\nsns.boxplot(x="listed_in(type)", y="approx_cost(for two people)", data=df\nplt.title("Boxplot of Approximate Cost (for two people) by Restaurant Typ\nplt.xlabel("Restaurant Type")\nplt.ylabel("Approx Cost (for two people)")\nplt.xticks(rotation=45, ha='right')\nplt.tight_layout()\nplt.show()
```

Data after cleaning:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 148 entries, 0 to 147

Data columns (total 7 columns):

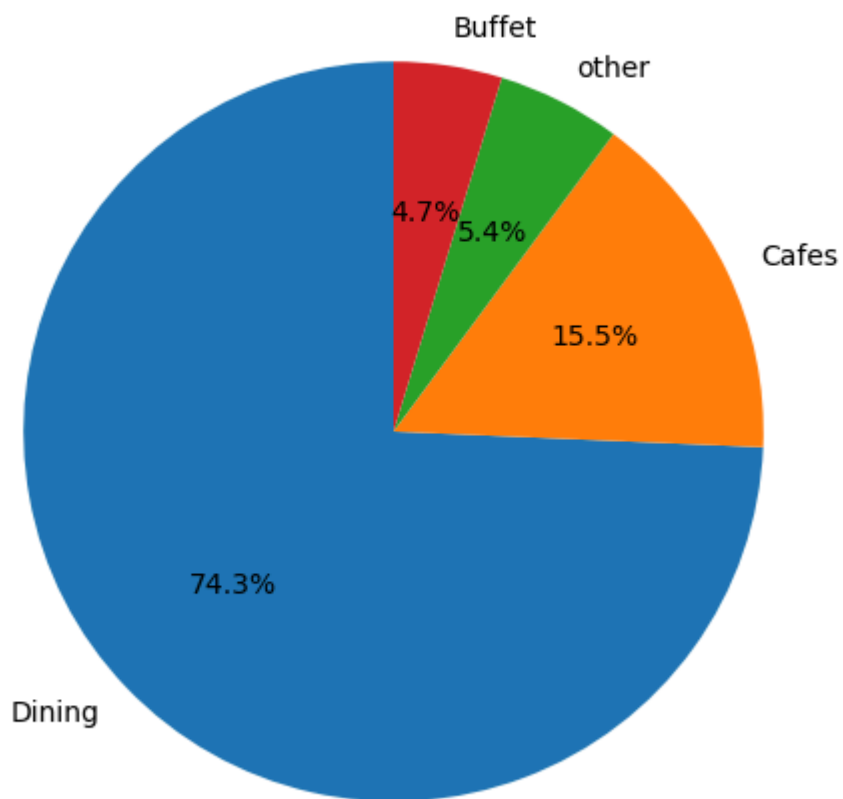
#	Column	Non-Null Count	Dtype
0	name	148 non-null	object
1	online_order	148 non-null	object
2	book_table	148 non-null	object
3	rate	148 non-null	object
4	votes	148 non-null	int64
5	approx_cost(for two people)	148 non-null	int64
6	listed_in(type)	148 non-null	object

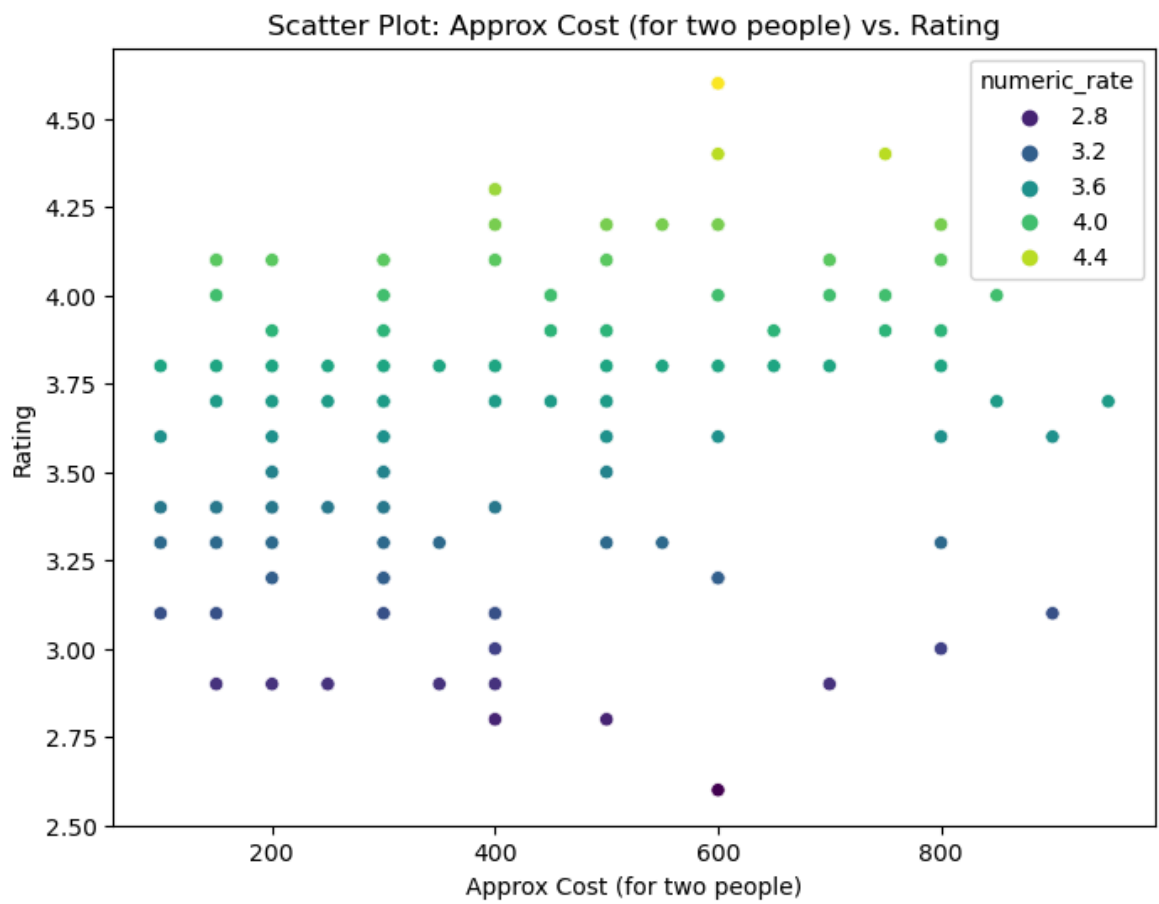
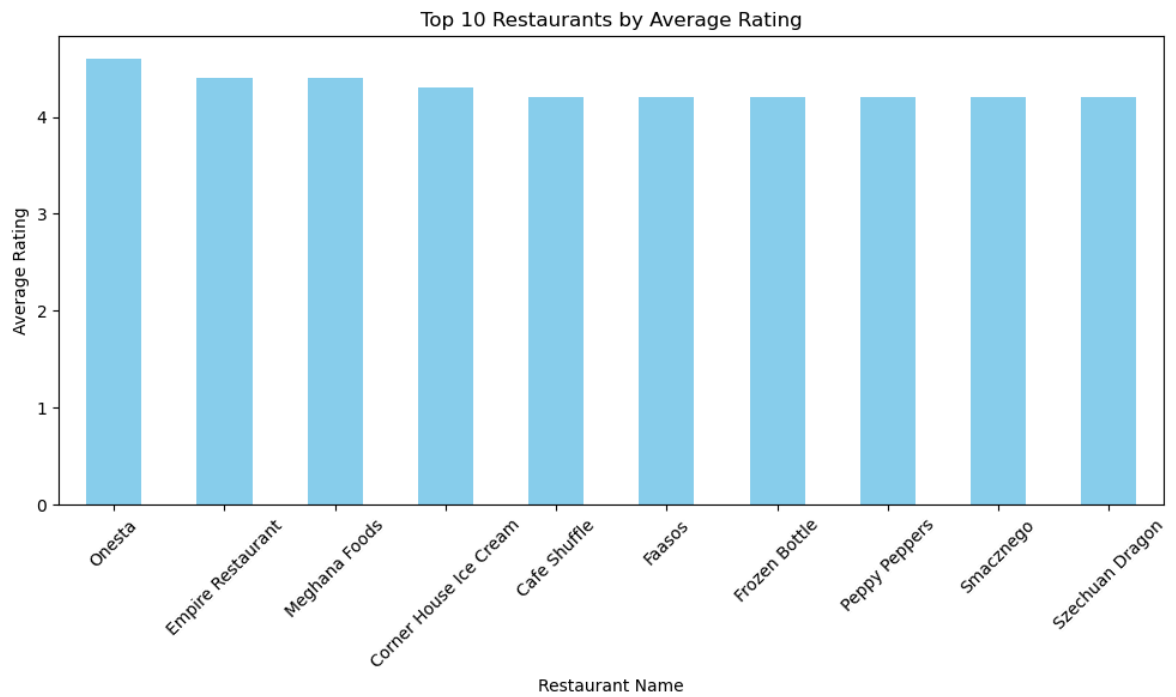
dtypes: int64(2), object(5)

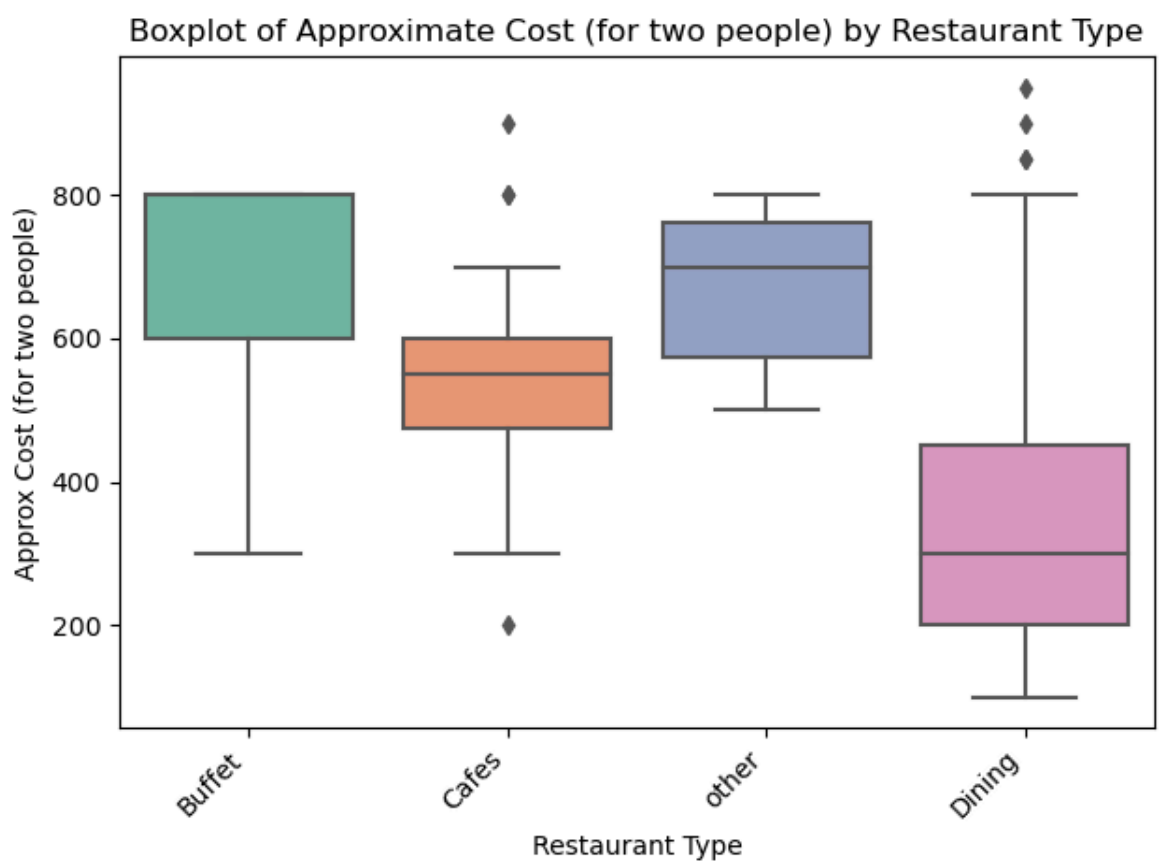
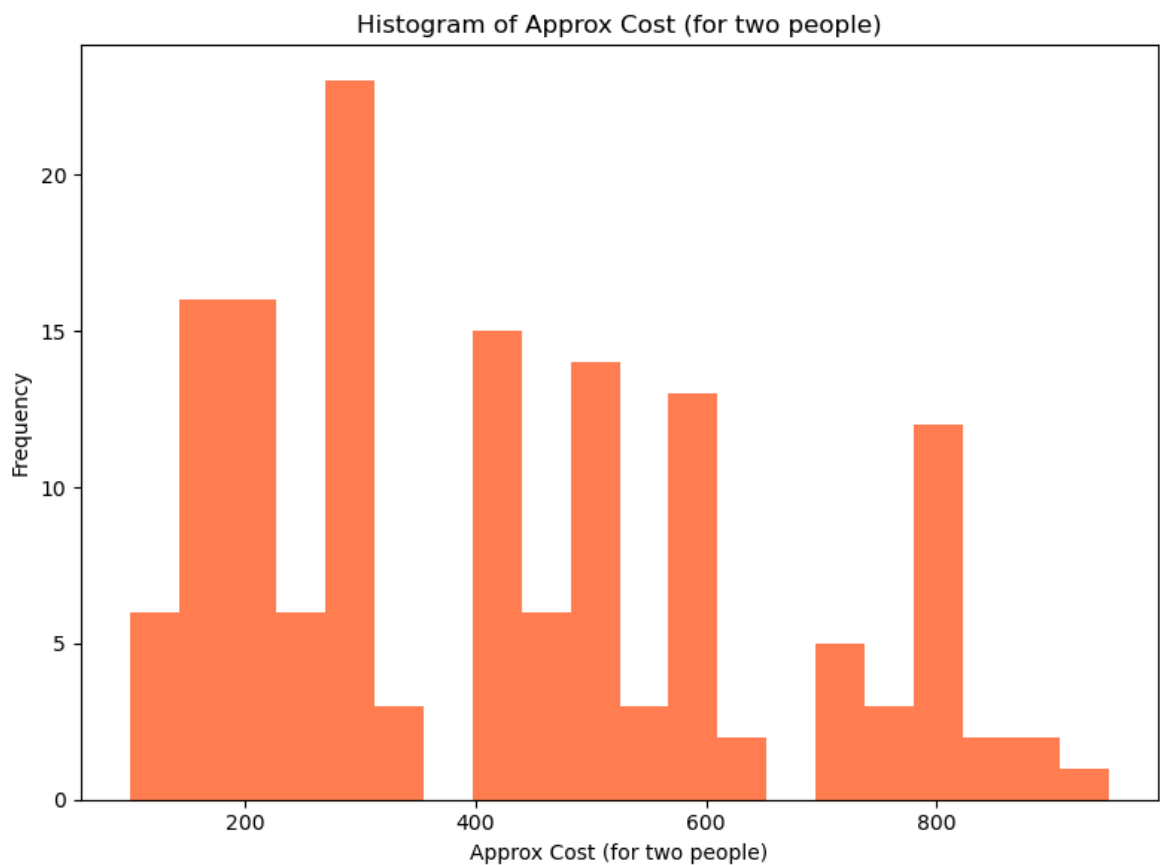
memory usage: 8.2+ KB

None

Pie Chart of listed_in(type)







In []:

In []: